# Introduction to Generative AI

Week 14

# Learning Goals

- Understand what Generative AI is
- See how Large Language Models (LLMs) work at a high level
- Practice prompting and see typical pitfalls (hallucinations)
- Discover how to build a simple generative AI assistant for event management
- Get a small assignment you can implement & test easily

# What is Generative AI?

- AI systems that generate new content: text, images, code, audio, video
- Trained on huge datasets; learns patterns and probabilities
- Generates outputs that were not explicitly seen during training

# Classic AI vs Generative AI

- Classic AI: rules, logic, search, optimization; outputs decisions/labels
- Examples: path planning, chess engines, spam filters
- Generative AI: predicts next token to create new content
- Examples: ChatGPT answers, code completion, story generators
- They are complementary, not competing approaches

# Large Language Models (LLMs)

- Neural networks trained on massive text corpora
- Learn statistical patterns of language
- Generate outputs token by token
- Key idea: Given previous text, predict the most likely next token

# What Can LLMs Do?

- Answer questions and explain concepts
- Summarize articles and papers
- Translate between languages
- Generate and complete code
- Role-play: tutor, event assistant, reviewer, etc.

# Prompting: Talking to LLMs

- A prompt is the instruction you give the model
- Good prompts specify a role, context, and desired format
- Prompt quality strongly affects output quality
- Think of prompts as programming with natural language

# Example: Prompt Improvement

- Bad: "Tell me about the conference."
- Better: "You are an assistant for the 'ICSE 2026' conference. In 5 bullet points, describe the event (date, location, main topics, target audience, registration deadline)."
- **Activity:** Propose ways to improve this prompt further (tone, length limits, etc.).

# Limitations of Generative AI

- No real understanding – pattern matching, not reasoning like humans
- Can hallucinate: invents facts, references, dates, locations
- Sensitive to prompt phrasing and hidden biases in training data
- Needs careful use and evaluation, especially in critical contexts

# Grounding the Model (RAG)

- Instead of letting the model invent answers, give it trusted data
- Use event schedule, venue info, FAQs as context
- Retrieval-Augmented Generation (RAG):
- 1. Retrieve relevant documents
- 2. Feed them into the prompt as context
- 3. Ask the LLM to answer only from that context

# From Chatbots to Agents

- Agent = LLM + goal + tools + memory/state
- Can call functions/APIs (e.g., check room availability)
- Can use a knowledge base (RAG) to stay grounded
- Can plan multi-step tasks to reach a goal
- In our context: an event management assistant agent

# Event Management AI projects

Your goal is to **design, implement, and document one AI agent** that supports a specific task of event management for the fictional conference **ICSE 2026.** All agents will share a common interface (defined in common/agent_base.py) and will be testable through a shared orchestrator (orchestrator/coordinator.py).

The conference at hand: https://conf.researchr.org/home/icse-2026

Github code: https://github.com/darkobozhinoski/event-management-agents

By completing this assignment, you will:

- Build a modular **AI agent** that solves a real event-management task
- Use a **Large Language Model** (via the provided llm_wrapper.py)
- Work with prompts, grounding, and structured outputs
- Understand how multiple AI agents can fit together in a larger system
- Practice collaborative software development using GitHub

# Technical Requirements



Each student creates at least **one agent** that:

- Extends a common Agent base class
- Takes a structured Python dict as input
- Returns a structured Python dict with:
  - success (bool)
  - message (str)
  - data (dict or None)

- Optionally uses the provided LLM wrapper
- Is registered in the shared coordinator.py

Each agent must:

1. Live in its own folder under agents/
   Implement:
   - name
   - description
   - run(request: dict) -> dict
2. Handle invalid inputs gracefully
   Return structured outputs
3. Include a README with examples
4. Be registered in orchestrator/coordinator.py

# Choose an Agent from this list



**Phase A — Pre-event**

- Call-for-Papers Assistant
- Sponsor Pitch Writer
- Talk/Track Classifier

**Phase B — Registration & Promotion**

- Registration FAQ Agent
- Marketing Content Generator

**Phase C — Logistics & Scheduling**

- Session Scheduler Helper
- Room/Capacity Checker
- Speaker Reminder Assistant

**Phase D — During Event**

- Speaker Reminder Assistant
- On-site Helpdesk Chatbot
- Issue Triage Agent

**Phase E — Post-event**

- Feedback Summarizer
- Certificate/Follow-up Email Generator

# Collaborative Development

Use this repo: https://github.com/darkobozhinoski/event-management-agents

- Each **student** works on **its own branch**:
  - No direct pushes to master
  - Only merge via **Pull Requests (PRs)**
  - Branch name convention: <student_name>-<agent-name>  e.g. `Darko-feedback-summarizer`
- Each student:
  - Only edits their own folder under `agents/…`
  - Adds their agent once into `orchestrator/coordinator.py`
  - Opens a PR when ready

# Recap & Next Steps

- Generative AI creates new content using learned patterns
- LLMs are powerful but can hallucinate and must be grounded